2HỆ THỐNG NHÚNG · BÀI THỰC HÀNH 1

Làm việc với LED — Bài tập

Phiếu bài tập & mẫu nộp bài — Zephyr RTOS trên Kit BKIT ARM4 (STM32F407ZGT6)

Môn học: Hệ thống nhúng · **Nền tảng:** Kit BKIT ARM4 (STM32F407ZGT6) · **Framework:** Zephyr RTOS v4.4.0 + VS Code

Thông tin sinh viên

Thông tin	Nội dung
Họ và tên	PHẠM CÔNG VÕ
MSSV	2313946
Lớp / Nhóm	L02
Ngày thực hiện	23/10/2026

Cách làm và nộp bài

File này vừa là đề bài vừa là mẫu nộp bài cho Bài thực hành 1. Đọc kỹ các bước sau trước khi bắt đầu:

1. Điền thông tin sinh viên vào bảng trên (họ tên, MSSV, lớp, ngày).
2. Mỗi bài tập, sao chép project `bkit_blinky` (Phần C của tài liệu hướng dẫn) thành một project mới, ví dụ `lab1_bai1`, `lab1_bai2`, … trong thư mục `zephyrproject\applications\`. Giữ nguyên `app.overlay`, `prj.conf`, `CMakeLists.txt` và `.vscode`; chỉ viết lại `src/main.c`.
3. Giải từng bài **trên phần cứng thật**: build bằng `west build`, nạp bằng `west flash` và xác nhận kit chạy đúng yêu cầu.
4. Dán **toàn bộ** file `src/main.c` (gồm các `#include`, khai báo, hàm thread, `K_THREAD_DEFINE`, `main()`) vào ô trả lời ngay bên dưới bài đó. Nếu bạn có sửa `app.overlay` hoặc `prj.conf`, dán thêm cả phần đã sửa.
5. Giữ code dễ đọc: thụt lề đúng, mỗi đèn do **một thread riêng** điều khiển, đặt tên thread theo đúng tên trong đề (ví dụ `thread_led_blinky`, `thread_y0_blinky`).
6. Không xoá hay thu nhỏ các ô trả lời — ô sẽ tự giãn khi bạn dán thêm dòng.
7. Hoàn thành cả năm bài, lưu file (giữ dạng .docx hoặc xuất PDF) và nộp **toàn bộ tài liệu**.

> **Nhắc lại — các đèn dùng trong bài tập**
>
> Ba đèn đã được khai báo sẵn trong `app.overlay` của project mẫu:
>
> - **LED3 (LED DEBUG)** — chân **PE4** — alias `led0`
> - **LED trạng thái Output Y0** — chân **PE5** — alias `led1`
> - **LED trạng thái Output Y1** — chân **PE6** — alias `led2`
>
> Lấy thông tin chân trong code bằng: `static const struct gpio_dt_spec y0 = GPIO_DT_SPEC_GET(DT_ALIAS(led1), gpios);`
>
> Các ngõ ra Y0/Y1 đi qua opto ra cầu đấu 12V. **Không đấu tải vào cầu đấu Output** trong bài này — chỉ quan sát LED trạng thái. Nếu LED trạng thái sáng ngược với mong đợi, đổi

`GPIO_ACTIVE_HIGH` thành `GPIO_ACTIVE_LOW` cho node tương ứng trong overlay và ghi chú lại trong bài nộp.

Gợi ý — khung chương trình nhiều thread

Mỗi thread là một hàm `void ten_thread(void *p1, void *p2, void *p3)` chứa vòng lặp `while (1)` vô hạn và dùng `k_msleep()` để chờ. Thread được tạo ở phạm vi toàn cục bằng `K_THREAD_DEFINE(id, 1024, ten_thread, NULL, NULL, NULL, 7, 0, 0);`. Hàm `main()` có thể chỉ `return 0;` — các thread vẫn tiếp tục chạy.

Tham khảo bảng so sánh FreeRTOS ↔ Zephyr ở Mục 7 của tài liệu hướng dẫn.

Ghi chú chấm điểm

Bạn được chấm dựa trên hành vi đúng trên phần cứng và mã nguồn sạch, đầy đủ, dán đúng ô. Ô trống, hoặc code không đúng yêu cầu của bài, sẽ không được tính điểm.

BÀI 1 Nháy LED với thời gian sáng/tắt khác nhau

Mục tiêu: Cấu trúc thread cơ bản và định thời với `k_msleep()`.

Yêu cầu

- Viết thread `thread_led_blinky` điều khiển **LED3**: **sáng 1 giây**, **tắt 2 giây**, lặp lại liên tục.
- Chỉ chạy **duy nhất thread này** (tạo bằng `K_THREAD_DEFINE`); hàm `main()` không điều khiển LED.
- Dùng `gpio_pin_set_dt()` (không dùng `gpio_pin_toggle_dt()`) để trạng thái sáng/tắt rõ ràng trong code.

Bài àm — toàn bộ mã nguồn:

Bài 1 – src/main.c

```
#include <stdio.h>
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>

#define TIME_LED_ON_MS   1000   /* Thời gian đèn sáng: 1000 ms = 1 giây */
#define TIME_LED_OFF_MS  2000   /* Thời gian đèn tắt: 2000 ms = 2 giây */

#define BLINKY_STACK_SIZE 1024
#define BLINKY_PRIORITY   7

/* Lấy thông tin cấu hình LED3 từ Devicetree alias "led0" */
#define LED3_NODE DT_ALIAS(led0)
static const struct gpio_dt_spec led3 = GPIO_DT_SPEC_GET(LED3_NODE, gpios);
```

```c
/* HÀM THỰC THI CỦA THREAD (Thread Entry Function): */
void thread_led_blinky_entry(void *p1, void *p2, void *p3)
{
    ARG_UNUSED(p1); ARG_UNUSED(p2); ARG_UNUSED(p3);

    /* 1. Kiểm tra driver GPIO của Port E đã khởi tạo và sẵn sàng chưa */
    if (!gpio_is_ready_dt(&led3)) {
        printf("[Lỗi] GPIO LED3 chưa sẵn sàng!\n");
        return;
    }

    /* 2. Cấu hình chân PE4 làm ngõ ra (Output), trạng thái ban đầu TẮT */
    gpio_pin_configure_dt(&led3, GPIO_OUTPUT_INACTIVE);

    printf("[Lab 1 - Bài 1] Bắt đầu: SÁNG %d ms --> TẮT %d ms\n",
TIME_LED_ON_MS, TIME_LED_OFF_MS);

    while (1) {
        /* BẬT LED3: Xuất mức 1 (3.3V) -> LED SÁNG */
        gpio_pin_set_dt(&led3, 1);
        printf("[%6u ms] [LED3] SÁNG (1s)\n", k_uptime_get_32());
        k_msleep(TIME_LED_ON_MS); /* Ngủ 1s nhường CPU */

        /* TẮT LED3: Xuất mức 0 (0V) -> LED TẮT */
        gpio_pin_set_dt(&led3, 0);
        printf("[%6u ms] [LED3] TẮT  (2s)\n", k_uptime_get_32());
        k_msleep(TIME_LED_OFF_MS); /* Ngủ 2s nhường CPU */
    }
}

/*
================================================================================
 * ĐĂNG KÝ VÀ KHỞI TẠO THREAD TĨNH (K_THREAD_DEFINE):
================================================================================
 */
K_THREAD_DEFINE(thread_led_blinky, BLINKY_STACK_SIZE,
                thread_led_blinky_entry, NULL, NULL, NULL,
                BLINKY_PRIORITY, 0, 0);

int main(void)
{
    printf("[Main] Bài 1 hoàn tất khởi động. Thread chạy ngầm.\n");
    return 0;
}
```

BÀI 2 Đồng bộ LED3 và Output Y0

Mục tiêu: Phối hợp giữa hai thread thông qua biến dùng chung.

Yêu cầu

- Viết `thread_led_blinky` điều khiển **LED3** theo chu kỳ **sáng 2 giây, tắt 2 giây**.
- Viết `thread_y0_follow` điều khiển **LED trạng thái Y0**: **sáng khi LED3 sáng** và **tắt khi LED3 tắt**.
- Hai thread phối hợp qua **một biến dùng chung** (ví dụ `atomic_t led_state` hoặc `volatile bool`); `thread_y0_follow` chỉ đọc biến này, không tự đếm thời gian 2 giây.
- `thread_y0_follow` kiểm tra biến định kỳ (ví dụ mỗi 10 ms) và cập nhật Y0 khi trạng thái thay đổi.

Bài làm — toàn bộ mã nguồn:

Bài 2 – src/main.c

```
#include <stdio.h>
#include <stdbool.h>
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>

#define STACK_SIZE 1024         /* Kích thước stack cho mỗi thread: 1 KB */
#define PRIORITY   7            /* Cả 2 thread cùng chạy ở độ ưu tiên 7 */

#define LED3_NODE DT_ALIAS(led0) /* LED3 (PE4) */
#define Y0_NODE   DT_ALIAS(led1) /* Output Y0 (PE5) */

static const struct gpio_dt_spec led3 = GPIO_DT_SPEC_GET(LED3_NODE, gpios);
static const struct gpio_dt_spec y0   = GPIO_DT_SPEC_GET(Y0_NODE, gpios);

/* Biến dung chung đồng bộ hay Thread */
static volatile bool g_led_state = false;

/*
 ================================================================================
 * HÀM THỰC THI CỦA THREAD 1: thread_led_blinky_entry
 ================================================================================
 */
void thread_led_blinky_entry(void *p1, void *p2, void *p3)
{
    ARG_UNUSED(p1); ARG_UNUSED(p2); ARG_UNUSED(p3);

    /* Kiểm tra và cấu hình GPIO LED3 */
    if (!gpio_is_ready_dt(&led3)) return;
    gpio_pin_configure_dt(&led3, GPIO_OUTPUT_INACTIVE);

    while (1) {
        /* Bật LED3 và ghi nhận biến trạng thái = true (SÁNG) */
        gpio_pin_set_dt(&led3, 1);
        g_led_state = true;
        printf("[%6u ms] [Master LED3] SÁNG (2s)\n", k_uptime_get_32());
        k_msleep(2000); /* Ngủ 2 giây nhường CPU */

        /* Tắt LED3 và ghi nhận biến trạng thái = false (TẮT) */
        gpio_pin_set_dt(&led3, 0);
        g_led_state = false;
        printf("[%6u ms] [Master LED3] TẮT  (2s)\n", k_uptime_get_32());
        k_msleep(2000); /* Ngủ 2 giây nhường CPU */
```

```
    }
}

void thread_y0_follow_entry(void *p1, void *p2, void *p3)
{
    ARG_UNUSED(p1); ARG_UNUSED(p2); ARG_UNUSED(p3);

    /* Kiểm tra và cấu hình GPIO Output Y0 */
    if (!gpio_is_ready_dt(&y0)) return;
    gpio_pin_configure_dt(&y0, GPIO_OUTPUT_INACTIVE);

    bool last_state = false; /* Lưu lại trạng thái của lần thăm dò liền trước
*/

    while (1) {
        bool current_state = g_led_state;

        /*Nếu phát hiện trạng thái có sự thay đổi->Đồng bộ Output Y0 theo */
        if (current_state != last_state) {
            gpio_pin_set_dt(&y0, current_state ? 1 : 0);
            printf("[%6u ms] >>> [Follower Y0] Cập nhật theo LED3: %s\n",
                   k_uptime_get_32(), current_state ? "SÁNG" : "TẮT");
            last_state = current_state;
        }

        k_msleep(10);
    }
}

/*
 ================================================================================
 * ĐĂNG KÝ 2 THREAD TĨNH VỚI ZEPHYR (K_THREAD_DEFINE):
 ================================================================================
 */
K_THREAD_DEFINE(thread_led_blinky, STACK_SIZE,
                thread_led_blinky_entry, NULL, NULL, NULL,
                PRIORITY, 0, 0);

K_THREAD_DEFINE(thread_y0_follow, STACK_SIZE,
                thread_y0_follow_entry, NULL, NULL, NULL,
                PRIORITY, 0, 0);

int main(void)
{
    printf("[Main] Bài 2 hoàn tất khởi động. 2 Thread đang đồng bộ qua biến
chung.\n");
    return 0;
}
```

BÀI 3 Các mẫu nháy độc lập

Mục tiêu: Thực hành nhiều chu kỳ định thời chạy bất đồng bộ, độc lập với nhau.

Yêu cầu

- Viết `thread_led_blinky` điều khiển **LED3** theo chu kỳ **sáng 1,5 giây / tắt 0,5 giây**.
- Viết `thread_output_sequence` điều khiển cặp **Y0, Y1** lần lượt qua ba trạng thái, luôn luôn thay đổi:
  - **Chỉ Y0 sáng** trong 1 giây
  - **Chỉ Y1 sáng** trong 2 giây
  - **Cả Y0 và Y1 cùng sáng** trong 2 giây, rồi quay lại trạng thái đầu
- Hai thread phải chạy **hoàn toàn độc lập**, không phụ thuộc trạng thái của nhau (không dùng biến dùng chung).

Bài làm — toàn bộ mã nguồn:

Bài 3 – src/main.c

```
#include <stdio.h>
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>

#define STACK_SIZE 1024         /* Stack 1024 bytes cho mỗi thread */
#define PRIORITY   7            /* Cùng độ ưu tiên 7 trong Scheduler */

#define LED3_NODE DT_ALIAS(led0)  /* PE4 */
#define Y0_NODE   DT_ALIAS(led1)   /* PE5 */
#define Y1_NODE   DT_ALIAS(led2)  /* PE6 */

static const struct gpio_dt_spec led3 = GPIO_DT_SPEC_GET(LED3_NODE, gpios);
static const struct gpio_dt_spec y0   = GPIO_DT_SPEC_GET(Y0_NODE, gpios);
static const struct gpio_dt_spec y1   = GPIO_DT_SPEC_GET(Y1_NODE, gpios);

/*
================================================================================
 HÀM THỰC THI CỦA THREAD 1: thread_led_blinky_entry (LUỒNG LED3)
================================================================================
 */
void thread_led_blinky_entry(void *p1, void *p2, void *p3)
{
    ARG_UNUSED(p1); ARG_UNUSED(p2); ARG_UNUSED(p3);

    if (!gpio_is_ready_dt(&led3)) return;
    gpio_pin_configure_dt(&led3, GPIO_OUTPUT_INACTIVE);

    while (1) {
        /* BẬT LED3 trong 1.5 giây */
        gpio_pin_set_dt(&led3, 1);
        printf("[%6u ms] [LED3] SÁNG (1.5s)\n", k_uptime_get_32());
        k_msleep(1500);

        /* TẮT LED3 trong 0.5 giây */
        gpio_pin_set_dt(&led3, 0);
        printf("[%6u ms] [LED3] TẮT  (0.5s)\n", k_uptime_get_32());
        k_msleep(500);
    }
}

/*
================================================================================
 * HÀM THỰC THI CỦA THREAD 2: thread_output_sequence_entry
================================================================================
 */
void thread_output_sequence_entry(void *p1, void *p2, void *p3)
{
    ARG_UNUSED(p1); ARG_UNUSED(p2); ARG_UNUSED(p3);

    if (!gpio_is_ready_dt(&y0) || !gpio_is_ready_dt(&y1)) return;
    gpio_pin_configure_dt(&y0, GPIO_OUTPUT_INACTIVE);
    gpio_pin_configure_dt(&y1, GPIO_OUTPUT_INACTIVE);

    while (1) {
        /* PHA 1: Chỉ Y0 sáng trong 1 giây (Y0=1, Y1=0) */
        gpio_pin_set_dt(&y0, 1);
        gpio_pin_set_dt(&y1, 0);
        printf("[%6u ms] [Y0/Y1] Pha 1: Chỉ Y0 SÁNG (1s)\n", k_uptime_get_32());
        k_msleep(1000);

        /* PHA 2: Chỉ Y1 sáng trong 2 giây (Y0=0, Y1=1) */
        gpio_pin_set_dt(&y0, 0);
        gpio_pin_set_dt(&y1, 1);
        printf("[%6u ms] [Y0/Y1] Pha 2: Chỉ Y1 SÁNG (2s)\n", k_uptime_get_32());
        k_msleep(2000);

        /* PHA 3: Cả Y0 và Y1 cùng sáng trong 2 giây (Y0=1, Y1=1) */
```

```
        gpio_pin_set_dt(&y0, 1);
        gpio_pin_set_dt(&y1, 1);
        printf("[%6u ms] [Y0/Y1] Pha 3: Cả Y0 và Y1 SÁNG (2s)\n", k_uptime_get_32());
        k_msleep(2000);
    }
}

/*
================================================================================
 * ĐĂNG KÝ 2 THREAD TĨNH (K_THREAD_DEFINE):
================================================================================
 */
K_THREAD_DEFINE(thread_led_blinky, STACK_SIZE,
                thread_led_blinky_entry, NULL, NULL, NULL,
                PRIORITY, 0, 0);

K_THREAD_DEFINE(thread_output_sequence, STACK_SIZE,
                thread_output_sequence_entry, NULL, NULL, NULL,
                PRIORITY, 0, 0);

int main(void)
{
    printf("[Main] Bài 3 hoàn tất khởi động. 2 Thread chạy chu kỳ độc lập.\n");
    return 0;
}
```

BÀI 4 Một hàm thread – nhiều đèn (truyền tham số cho thread)

Mục tiêu: Hành vi phức tạp hơn và tham số hoá thread.

Yêu cầu

- Định nghĩa một struct cấu hình, ví dụ `struct blink_cfg { struct gpio_dt_spec led; int on_ms; int off_ms; };`.
- Viết **duy nhất một** hàm thread `thread_blink(void *p1, void *p2, void *p3)` nhận con trỏ tới `struct blink_cfg` qua tham số `p1`.
- Dùng `K_THREAD_DEFINE` **ba lần** với cùng hàm `thread_blink` nhưng tham số khác nhau:
  - **LED3**: sáng 2 giây, tắt 2 giây
  - **Y0**: sáng 0,5 giây, tắt 0,5 giây
  - **Y1**: sáng 3 giây, tắt 1 giây
- Cả ba thread chạy **đồng thời** và **không đồng bộ** với nhau. Trong bài nộp, giải thích ngắn (2–3 câu, dưới dạng chú thích trong code) vì sao không cần ba hàm thread riêng.

Bài làm — toàn bộ mã nguồn:

Bài 4 – src/main.c

```
#include <stdio.h>
#include <zephyr/drivers/gpio.h>
#include <zephyr/kernel.h>

#define STACK_SIZE 1024 /* Stack riêng 1024 bytes cho mỗi thread */
#define PRIORITY 7      /* Cả 3 thread cùng độ ưu tiên 7 */

#define LED3_NODE DT_ALIAS(led0) /* PE4 */
#define Y0_NODE DT_ALIAS(led1)   /* PE5 */
#define Y1_NODE DT_ALIAS(led2)   /* PE6 */

/* Cấu trúc dữ liệu chứa tham số cấu hình riêng cho từng đèn */
struct blink_cfg {
  struct gpio_dt_spec led; /* Thông tin phần cứng GPIO */
  int on_ms;               /* Thời gian sáng (ms) */
  int off_ms;              /* Thời gian tắt (ms) */
  const char *name;        /* Tên đại diện in log */
};

/* Khởi tạo tĩnh 3 bảng cấu hình riêng biệt trong bộ nhớ Flash (const) */
static const struct blink_cfg cfg_led3 = {
    .led = GPIO_DT_SPEC_GET(LED3_NODE, gpios),
    .on_ms = 2000,
    .off_ms = 2000,
    .name = "LED3"};

static const struct blink_cfg cfg_y0 = {
     .led = GPIO_DT_SPEC_GET(Y0_NODE, gpios),
     .on_ms = 500,
     .off_ms = 500,
     .name = "OUT_Y0"};

static const struct blink_cfg cfg_y1 = {
     .led = GPIO_DT_SPEC_GET(Y1_NODE, gpios),
     .on_ms = 3000,
     .off_ms = 1000,
     .name = "OUT_Y1"};

/*
 * ======================================================================
 * HÀM THỰC THI CHUNG DUY NHẤT: thread_blink (GENERIC THREAD ENTRY FUNCTION)
 * ======================================================================
 */
void thread_blink(void *p1, void *p2, void *p3) {
  ARG_UNUSED(p2);
  ARG_UNUSED(p3);

  // ÉP KIỂU CON TRỎ (TYPE CASTING):
  const struct blink_cfg *cfg = (const struct blink_cfg *)p1;

  if (!gpio_is_ready_dt(&cfg->led))
    return;
  gpio_pin_configure_dt(&cfg->led, GPIO_OUTPUT_INACTIVE);

  while (1) {
    /* Bật đèn theo thời gian on_ms */
    gpio_pin_set_dt(&cfg->led, 1);
    printf("[%6u ms] [%-6s] SÁNG (%d ms)\n", k_uptime_get_32(), cfg->name,
           cfg->on_ms);
    k_msleep(cfg->on_ms);

    /* Tắt đèn theo thời gian off_ms */
    gpio_pin_set_dt(&cfg->led, 0);
    printf("[%6u ms] [%-6s] TẮT  (%d ms)\n", k_uptime_get_32(), cfg->name,
           cfg->off_ms);
    k_msleep(cfg->off_ms);
  }
```

```
}

/*
 * ========================================================================
 * ĐĂNG KÝ 3 THREAD TĨNH CÙNG TRỎ VÀO MỘT HÀM thread_blink:
 * ========================================================================
 */
K_THREAD_DEFINE(t_blink_led3, STACK_SIZE, thread_blink, (void *)&cfg_led3, NULL,
                NULL, PRIORITY, 0, 0);
K_THREAD_DEFINE(t_blink_y0, STACK_SIZE, thread_blink, (void *)&cfg_y0, NULL,
                NULL, PRIORITY, 0, 0);
K_THREAD_DEFINE(t_blink_y1, STACK_SIZE, thread_blink, (void *)&cfg_y1, NULL,
                NULL, PRIORITY, 0, 0);

int main(void) {
  printf(
      "[Main] Bài 4 hoàn tất khởi động. 3 Thread dùng chung 1 hàm thực thi.\n");
  return 0;
}



/*
 * ========================================================================
 * TRẢ LỜI CÂU HỎI: VÌ SAO KHÔNG CẦN BA HÀM THREAD RIÊNG?
 * ------------------------------------------------------------------------
 * Bản chất logic thực thi của cả ba đèn là hoàn toàn đồng nhất: Đều lặp vô hạn
 * chu trình [Bật đèn -> k_msleep(on_ms) -> Tắt đèn -> k_msleep(off_ms)].
 * Điểm khác biệt duy nhất giữa chúng chỉ là tham số dữ liệu (chân GPIO nào và
 * thời gian sáng/tắt bao lâu).
 *
 * Nếu viết ba hàm riêng biệt, mã nhị phân sinh ra sẽ bị trùng lặp ba lần,
 * gây lãng phí bộ nhớ Flash của vi điều khiển.
 *
 * Bằng cách gom dữ liệu vào struct cấu hình và truyền qua con trỏ tham số p1 của
 * K_THREAD_DEFINE, cả ba thread dùng chung một đoạn mã lệnh (Code Segment) nhưng
 * mỗi thread sở hữu một vùng nhớ Stack và thanh ghi ngữ cảnh độc lập, giúp mã
 * nguồn tối ưu bộ nhớ, cô đọng và dễ mở rộng.
 * ========================================================================
 */
```

BÀI 5 Mô phỏng hệ thống đèn giao thông

Mục tiêu: Mô phỏng bộ điều khiển đèn giao thông đơn giản bằng các thread bất đồng bộ.

Các thread cần viết

- **Thread 1** (`thread_led_blinky`): LED3 nháy liên tục — **sáng 1 giây, tắt 1 giây** — báo hiệu hệ thống đang chạy.
- **Thread 2** (`thread_traffic_light`): cặp Y0/Y1 biểu diễn đèn giao thông, quy ước **Y0 = ĐỎ**, **Y1 = XANH**, **cả hai cùng sáng = VÀNG**. Lặp vô hạn chu kỳ:
  - **ĐỎ** (chỉ Y0 sáng) trong **7 giây**
  - **XANH** (chỉ Y1 sáng) trong **5 giây**
  - **VÀNG** (Y0 + Y1 cùng sáng) trong **2 giây**, rồi quay lại ĐỎ
- Mỗi khi đổi pha, in ra console tên pha và thời gian, ví dụ `printf("[DEN] DO - 7 s\n");`. (Có cáp USB–RS232 thì chụp màn hình Serial Monitor dán kèm — điểm cộng.)
- Lưu thời lượng của ba pha trong **một mảng hằng** (ví dụ `struct phase { const char *name; int y0; int y1; int ms; }`) và duyệt mảng trong vòng lặp, thay vì viết ba khối lệnh lặp lại.

Bài làm — toàn bộ mã nguồn:

Bài 5 – src/main.c

```
#include <stdio.h>
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>

#define STACK_SIZE 1024
#define PRIORITY   7

#define LED3_NODE DT_ALIAS(led0) /* PE4 - Heartbeat LED */
#define Y0_NODE   DT_ALIAS(led1) /* PE5 - Output Y0 */
#define Y1_NODE   DT_ALIAS(led2) /* PE6 - Output Y1 */

static const struct gpio_dt_spec led3 = GPIO_DT_SPEC_GET(LED3_NODE, gpios);
static const struct gpio_dt_spec y0   = GPIO_DT_SPEC_GET(Y0_NODE, gpios);
static const struct gpio_dt_spec y1   = GPIO_DT_SPEC_GET(Y1_NODE, gpios);

/* Cấu trúc dữ liệu đại diện cho một pha của đèn giao thông */
struct phase {
    const char *name;     /* Tên hiển thị của pha ("DO", "XANH", "VANG") */
    int y0;                 /* Trạng thái ngõ ra Y0 (1: Bật, 0: Tắt) */
    int y1;                 /* Trạng thái ngõ ra Y1 (1: Bật, 0: Tắt) */
    int ms;                /* Thời gian duy trì pha (mili-giây) */
};

/* Bảng dữ liệu cấu hình 3 pha đèn giao thông */
static const struct phase traffic_phases[] = {
    {
        .name = "DO",
        .y0   = 1,
        .y1   = 0,
        .ms   = 7000
    },
    {
        .name = "XANH",
        .y0   = 0,
        .y1   = 1,
        .ms   = 5000
    },
    {
        .name = "VANG",
        .y0   = 1,
        .y1   = 1,
        .ms   = 2000
    }
};

#define NUM_PHASES (sizeof(traffic_phases) / sizeof(traffic_phases[0]))

/*
 *
==========================================================================
 * HÀM THỰC THI CỦA THREAD 1: thread_led_blinky_entry (LUỒNG HEARTBEAT HỆ
THỐNG)
 *
==========================================================================
 */
void thread_led_blinky_entry(void *p1, void *p2, void *p3)
{
    ARG_UNUSED(p1); ARG_UNUSED(p2); ARG_UNUSED(p3);

    if (!gpio_is_ready_dt(&led3)) return;
    gpio_pin_configure_dt(&led3, GPIO_OUTPUT_INACTIVE);

    while (1) {
        /* Bật LED3 trong 1 giây */
        gpio_pin_set_dt(&led3, 1);
        k_msleep(1000);
```

```
        /* Tắt LED3 trong 1 giây */
        gpio_pin_set_dt(&led3, 0);
        k_msleep(1000);
    }
}

/*
 *
==========================================================================
 * HÀM THỰC THI CỦA THREAD 2: thread_traffic_light_entry (
 * ------------------------------------------------------------------
 * - Duyệt tuần tự qua từng phần tử trong bảng traffic_phases[].
 * - Xuất trạng thái logic tương ứng ra ngõ ra Y0 và Y1.
 * - In thông báo ra màn hình console và gọi k_msleep(p->ms) để giữ nguyên
trạng thái.
 *
==========================================================================
 */
void thread_traffic_light_entry(void *p1, void *p2, void *p3)
{
    ARG_UNUSED(p1); ARG_UNUSED(p2); ARG_UNUSED(p3);

    if (!gpio_is_ready_dt(&y0) || !gpio_is_ready_dt(&y1)) return;
    gpio_pin_configure_dt(&y0, GPIO_OUTPUT_INACTIVE);
    gpio_pin_configure_dt(&y1, GPIO_OUTPUT_INACTIVE);

    while (1) {
        /* Duyệt tuần tự qua tất cả các pha được định nghĩa trong mảng */
        for (int i = 0; i < NUM_PHASES; i++) {
            const struct phase *p = &traffic_phases[i];

            /* Xuất trạng thái đèn ra cổng Output Y0 và Y1 */
            gpio_pin_set_dt(&y0, p->y0);
            gpio_pin_set_dt(&y1, p->y1);
            printf("[%6u ms] [DEN] %s - %d s\n", k_uptime_get_32(), p->name,
p->ms / 1000);

            /* Duy trì trạng thái pha tương ứng thời gian cài đặt */
            k_msleep(p->ms);
        }
    }
}

/*
==========================================================================
* ĐĂNG KÝ 2 THREAD TĨNH (K_THREAD_DEFINE):
==========================================================================
 */
K_THREAD_DEFINE(thread_led_blinky, STACK_SIZE,
                thread_led_blinky_entry, NULL, NULL, NULL,
                PRIORITY, 0, 0);

K_THREAD_DEFINE(thread_traffic_light, STACK_SIZE,
                thread_traffic_light_entry, NULL, NULL, NULL,
                PRIORITY, 0, 0);

int main(void)
{
    printf("[Main] Bài 5 hoàn tất khởi động. Hệ thống đèn giao thông đang hoạt
động.\n");
    return 0;
}
```

Màn hình Serial Monitor :

